



# Memory 用户手册

EIC7x 系列 AI 数字 SoC

---

Engineering Draft / Rev 0.8.1

2025-08-01

## 声明

版权所有©2024，北京奕斯伟计算技术有限公司及其关联公司（以下简称“ESWIN”）。保留所有权利。

ESWIN 在此软件中保留所有知识产权和专有权利。未经 ESWIN 授权，不得发布、复制、分发、转载、修改、改编、翻译或制作此软件的衍生作品，无论是全部还是部分。

对于本文档的任何修改，ESWIN 无需承担通知义务且不代表 ESWIN 做出任何承诺。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证与承诺，无论明示或默示。（本段落用于文档中）

“ESWIN” logo 和其他 ESWIN 标识，为北京奕斯伟计算技术有限公司及（或）其母公司所有的商标。本文档提及的其他所有商标或商品名称，由其各自的所有权人所有。

产品安装中可能包含开源软件和/或自由软件的许可声明。

## Change History

| Version | Date       | 描述                                                                                                                                                                                                                                                  |
|---------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| V0.5.1  | 2024-02-20 | Initial edition, an engineering draft.                                                                                                                                                                                                              |
| V0.5.2  | 2024-09-09 | 增加 1.1 小节 配置 MMZ 空间大小                                                                                                                                                                                                                               |
| V0.5.3  | 2024-09-11 | 修改副标题为"EIC7x 系列 AI 数字 SoC"                                                                                                                                                                                                                          |
| V0.5.4  | 2024-11-04 | 2.1 小节, ES_SYS_MemFlushCache 接口, 增加 cache 同步的使用说明                                                                                                                                                                                                   |
| V0.6.0  | 2025-03-20 | 2.1 小节, 增加 ES_SYS_MemFlushAll, ES_SYS_MemFlushAllByNid 接口<br>第 3 节, 增加 ES_SYS_MemFlushAll, ES_SYS_MemFlushAllByNid 接口定义, 修改 ES_SYS_RetrieveMemNodeByFd 和 ES_SYS_RetrieveMemNodeByVaddr 的 node ID 数据类型。<br>第 4 节, 新增 EIC770X_LOGICAL_MEM_NODE_E 类型定义 |
| V0.6.1  | 2025-05-13 | 2.1 小节, 增加 ES_SYS_MemFree 的使用说明<br>2.2 小节, 增加 ES_VB_FreeIOVA 的使用说明                                                                                                                                                                                  |
| V0.7.0  | 2025-06-24 | 2.2 小节和第 3 节, 新增 ES_VB_registerBlockRefCntNotifyCallback 接口说明和定义<br>第 6 节, proc 调试信息获取目录修改为/proc/eswin/vb                                                                                                                                           |
| V0.8.0  | 2025-07-17 | 第 2 节修改 API 简介<br>添加 2.3 小节, 添加用户态内存分配管理<br>第 3 节, 添加 ES_USER_Malloc_To_MenFd 接口说明和定义                                                                                                                                                               |
| V0.8.1  | 2025-08-01 | 第一节概述部分, 添加 nid 和 partition 支持上限的描述, 每个 die 至多支持 8 个 partition。                                                                                                                                                                                     |

# 目录

|                          |           |
|--------------------------|-----------|
| .....                    | 0         |
| <b>MEMORY 用户手册 .....</b> | <b>0</b>  |
| EIC7x 系列 AI 数字 SoC ..... | 0         |
| 目录 .....                 | III       |
| 表目录 .....                | IV        |
| 图目录 .....                | V         |
| <b>1. 概述 .....</b>       | <b>1</b>  |
| 1.1 配置 MMZ 空间大小 .....    | 2         |
| <b>2. API 简介 .....</b>   | <b>5</b>  |
| 2.1 通用内存分配接口 .....       | 5         |
| 2.2 VB 内存池管理 .....       | 8         |
| 2.3 用户态内存分配管理 .....      | 11        |
| <b>3. API 定义 .....</b>   | <b>11</b> |
| <b>4. 数据类型 .....</b>     | <b>31</b> |
| <b>5. ERRCODE .....</b>  | <b>34</b> |
| <b>6. 附录 .....</b>       | <b>36</b> |

---

## 表目录

|                       |    |
|-----------------------|----|
| 表 5-1 Error code..... | 34 |
|-----------------------|----|

图目录

图 1-1 Memory 模块框架 ..... 1

图 1-2 eic7700-evb.dts 示例..... 4

## 1. 概述

Memory 模块主要向媒体业务提供大块物理内存管理功能，负责内存缓存池、内存块的分配和回收，让物理内存资源在各个媒体处理模块中合理使用。Memory 模块的框架如下图所示：

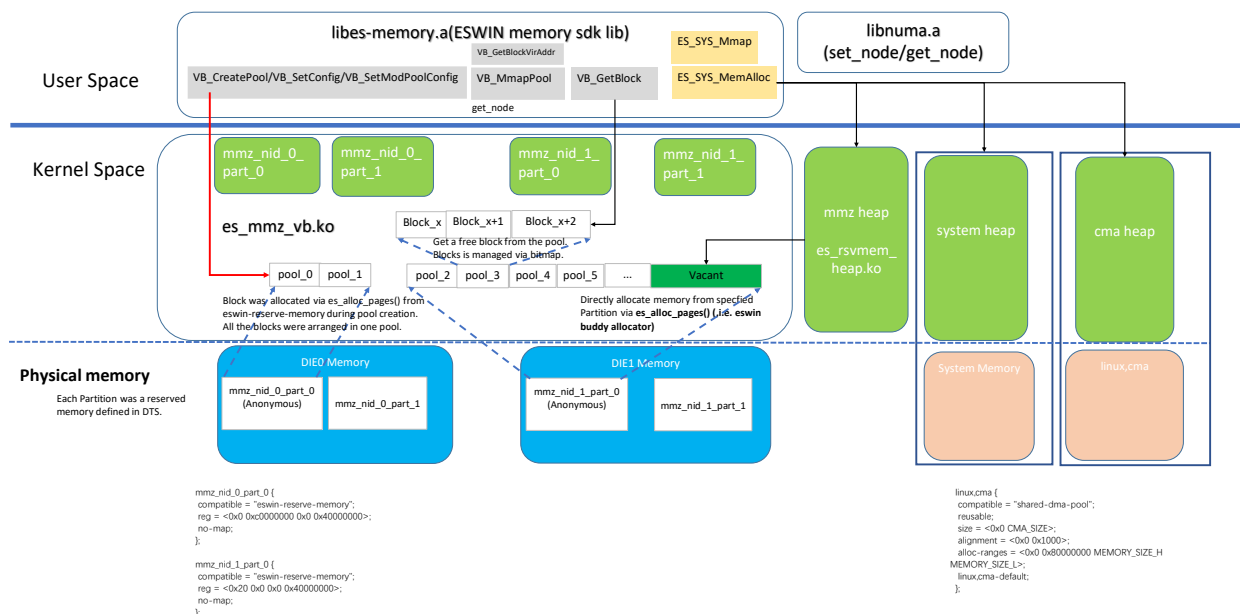


图 1-1 Memory 模块框架

Memory 功能模块所管理的内存空间被划分为以下几个区域，用户可以在这些区域分配内存块，每个被分配的内存块有一个引用计数，当引用计数为 0 时，该内存块会被释放回它所在的内存区域。

**mmz\_nid\_0\_part\_0:** die0 上编号为 0 的内存区域，在用户分配 die0 内存块时，如果没指定分配区域，此区域作为默认区域分配内存块。

**mmz\_nid\_0\_part\_1:** die0 上编号为 1 的内存区域。

**mmz\_nid\_1\_part\_0:** die1 上编号为 0 的内存区域，在用户分配 die1 内存块时，如果没指定分配区域，此区域作为默认区域分配内存块。

**mmz\_nid\_1\_part\_1:** die1 上编号为 1 的内存区域。

**llc\_spram0:** die0 上的 spram 内存区域。

**llc\_spram1:** die1 上的 spram 内存区域。

**system:** linux 系统自带的 dma heap 内存区域，在此区域分配的内存块物理地址不保证连续。

**linux,cma:** linux 系统自带的内存区域，在此区域分配的内存块物理地址连续。

注：目前至多两个 die，每个 die 至多支持 8 个 partition，也即 `mmz_nid_0_part_0`, `mmz_nid_0_part_1`, ..., `mmz_nid_0_part_7`, die1 也是。实际系统使用多少由 dts 配置决定，上限为 2 个 nid，每 nid 8 个 partition。

Memory 模块的用户态 lib 库为 `libes_memory.so`。

## 1.1 配置 MMZ 空间大小

对于 mmz\_nid\_x\_part\_x 内存区域，在此文中统一叫做 MMZ 的 partition。

MMZ partition，以及 linux,cma 都是 reserved memory 空间，其起始地址以及大小是通过 DTS 配置确定的。用户可根据实际板子的 DDR 容量大小，以及业务的需求，定义它们的起始地址和大小。注意，请避开 0xc0000000-0xc0200000 区域（该区域存放了 DTB）。如下 eic7700-evb.dts 所示：



```

#define MEMORY_SIZE_H    0x3
#define MEMORY_SIZE_L    0x80000000
#define CMA_SIZE         0x10000000

#include "eswin-win2030-die0-soc.dtsi"
#include "eic7700-pinctrl.dtsi"
#include <dt-bindings/interrupt-controller/irq.h>
#include <dt-bindings/gpio/gpio.h>

/* Clock frequency (in Hz) of the PCB crystal for rtcclk */

/ {
+   #address-cells = <2>;
+   #size-cells = <2>;
+   model = "ESWIN EIC7700";
+   compatible = "sifive,hifive-unmatched-a00", "sifive,fu740-c000",
+               "sifive,fu740", "eswin,eic7700";

+   aliases {
+       serial0 = &d0_uart0;
+       ethernet0 = &d0_gmac0;
+       ethernet1 = &d0_gmac1;
+   };

+   chosen {
+       stdout-path = "serial0:115200n8";
+   };

+   cpus {
+       timebase-frequency = <RTCCLK_FREQ>;
+   };
+
+   memory@80000000 {
+       device_type = "memory";
+       reg = <0x0 0x80000000 MEMORY_SIZE_H MEMORY_SIZE_L>;
+       numa-node-id = <0>;
+   };

+   reserved-memory {
+       #address-cells = <2>;
+       #size-cells = <2>;
+       ranges;

+       linux,cma {
+           compatible = "shared-dma-pool";
+           reusable;
+           size = <0x0 CMA_SIZE>;
+           alignment = <0x0 0x1000>;
+           alloc-ranges = <0x0 0x80000000 MEMORY_SIZE_H MEMORY_SIZE_L>;
+           linux,cma-default;
+       };
+
+   };

```



```

=> env print
arch=riscv
baudrate=115200
board=hifive_premier_p550
board_info=SF106SKB2425000128
board_name=hifive_premier_p550
bootcmd=run mmcboot;run bootkernel;
bootdelay=2
bootfile=SF106SKB2425000128/bootloader_uboot.bin
bootkernel=bootm 0x90000000
cpu=eic7700
dnsip=10.10.2.32
emmc_dev=0
ethladdr=94:47:b0:18:43:48
ethact=ethernet@50400000
ethaddr=94:47:b0:18:43:47
fdt_addr=ed916150
fdt_addr_r=0x88000000
fdt_high=0xffffffffffffffff
fdtaddr=ed916150
fdtcontroladdr=ed916150
fdtfile=eswin/eic7700-hifive-premier-p550.dtb
fileaddr=90000000
filesize=3c0108
gatewayip=10.10.192.1
gpt_partition=gpt write mmc ${emmc_dev} $partitions
initrd_high=0xffffffffffffffff
ipaddr=10.10.203.145
kernel_addr_r=0x84000000
kernel_comp_addr_r=0xa0000000
kernel_comp_size=0x4000000
loadaddr=0x80200000
mmcargs=setenv bootargs root=${mmcroot}
mmcboot=echo Booting from mmc ...;run mmcargs; run mmz_resize; run setimageload;run loadimage
mmcroot=/dev/mmcblk0p3 rootfstype=ext4 rootwait
mmz_resize=fdt mmz mmz nid 0 part 1 0x200000000 0x10000000

```

## 2. API 简介

提供三种内存分配方法：一种是通用内存分配接口，通过 `strZone` 参数选择内存是来自于 MMZ 的 partition（例如“`mmz_nid_0_part_0`”），或者“`system`”，“`linux,cma`”这样的 dma heap 区域，从这些区域分配任意指定大小的内存。第二种方法是从 MMZ 的 partition（例如“`mmz_nid_0_part_0`”），创建 pool，从 pool 中分配固定大小的 block 内存。该方法以下称之为 VB 内存池管理接口。第三种是通过用户 `malloc` 内存转换为 dma buf 内存。

### 2.1 通用内存分配接口

内存分配主要涉及以下 API：

#### ES\_SYS\_MemAlloc

从指定的内存区域分配内存 buffer。用户可以指定分配内存 buffer 的大小及内核态 cache 属性，也可以传入用户指定的内存 buffer 名称，用于 proc 信息呈现。

##### 【说明】

- 1、分配内存的区域名只能是前文中描述的内存区域。
- 2、调用内存分配接口分配出来的内存 buffer 用一个无符号 64 位类型的 `memFd` 表示，`memFd` 在进程内唯一。跨进程需要调用 `ES_SYS_RcvMemFd` 进行转换得到另外进程下对应的 `memFd`。

3、内存来自 llc\_spram0 时，内核态和用户态访问，都不会经过 cache，即 ES\_SYS\_MemAlloc 和 ES\_SYS\_Mmap 的 cacheMode 无效。

4、内存来自 llc\_spram0 时，分配的是物理地址非连续的 spram memory；来自 linux,cma 时，分配的是物理地址连续的 ddr memory；来自 mmz\_nid\_x\_part\_x 和 system 时，分配的是物理地址非连续的 ddr memory。

5、llc\_spram0 区域的大小为 4MB。

6、ESWIN memory 的设计基于 dma buf。

## ES\_SYS\_Mmap

映射指定内存 buffer，得到用户态虚拟地址。用户可以指定用户态 cache 属性。

### 【说明】

1、当通过 ES\_SYS\_MemAlloc 分配 SYS\_CACHE\_MODE\_LLC 属性的内存时，ES\_SYS\_Mmap 映射该内存的 cacheMode 参数必须也是 SYS\_CACHE\_MODE\_LLC，用户态 CPU 访问时，固定只能经过 LLC。

2、ES\_SYS\_Mmap 和 ES\_SYS\_Munmap 必须成对调用。调用 ES\_SYS\_Mmap 会使内存 buffer 引用计数 blockRefCount 加 1，调用 ES\_SYS\_Munmap 会使 blockRefCount 减 1。

## ES\_SYS\_Munmap

反映射指定内存 buffer 用户态虚拟地址。

## ES\_SYS\_MemFlushCache

刷新 cache 里的内容到内存并使 cache 里的内容无效。

### 【说明】

1、当通过 ES\_SYS\_Mmap 映射得到 CACHED\_BUF 属性的用户态虚拟地址，cpu 访问此虚拟地址，需调用 ES\_SYS\_MemFlushCache 做好数据同步。即：当 CPU 完成对该内存的写操作后，应调用 ES\_SYS\_MemFlushCache，以保证内容被写入内存；当 Device 完成对内存块的写入操作，CPU 读操作前，应调用 ES\_SYS\_MemFlushCache，以保证读到的内容与内存里的内容一致。

2、Flush cache 的范围是 memFd 指向的整个内存。

## ES\_SYS\_MemFree

释放通过 ES\_SYS\_MemAlloc 申请得到的内存 buffer。

### 【说明】

使用此接口释放 FD 之前，需要先释放该 FD 关联的 IOVA，否则后续 IOVA 无法释放会产生内存泄露。

## ES\_SYS\_SplitMemFd

从一个内存 buffer 的 offset 偏移处，按照指定的 len 大小，生成一个新的内存 buffer。

### 【说明】

- 1、不支持对 splitted memFd，再调用该 API 进一步 Split。
- 2、调用 ES\_SYS\_SplitMemFd，会使传入的 memFd 计数加 1，当 splittedMemFd 释放时，memFd 计数会自动减 1。
- 3、split

## ES\_SYS\_RcvMemFd

在不同进程或者同进程不同线程之间传递内存。

### 【说明】

- 1、接收到新的 rcvdMemFd，在使用完后，需要调用 ES\_SYS\_MemFree 或者 ES\_ReleaseBlock 接口释放。
- 2、所有内存分配接口得到的 memFd 都适用于 rcvdMemFd。
- 3、调用此接口，会使对应内存 buffer 的引用计数加 1。

## ES\_SYS\_GetMemSize

获取指定内存 buffer 的大小。

## ES\_SYS\_RetrieveMemNodeByFd

获取指定内存 buffer 所在的 node ID。

## ES\_SYS\_RetrieveMemNodeByVaddr

通过缓冲块虚拟地址获取缓存块所在的 node ID。

## ES\_SYS\_MemFlushCacheAllByNid

刷新指定 node ID cache 里的所有内容到内存并使 cache 里的内容无效。

### 【说明】

node ID 需使用 ES\_SYS\_RetrieveMemNodeByFd 或者 ES\_SYS\_RetrieveMemNodeByVaddr 接口获取，请不要随意指定数字。

## ES\_SYS\_MemFlushCacheAll

刷新所有 node ID cache 里的所有内容到内存并使 cache 里的内容无效。

### 【说明】

当编程人员无法确定 memory 来自哪一个 node ID 时使用此接口，性能相比指定 node id 可能会稍慢。

## 2.2 VB 内存池管理

内存池来自于 mmz\_nid\_x\_part\_x 区域，从内存池中申请的内存块是物理地址非连续的。内存池管理主要涉及以下 API：

### ES\_VB\_CreatePool

从指定的内存区域创建一个私有视频缓存池。

#### 【说明】

1、用户可以指定缓存池 pool 的属性，其中 enRemapMode 设置了 pool 在内核态的虚拟地址是 cache 还是 uncached。

### ES\_VB\_DestroyPool

销毁一个私有视频缓存池。

#### 【说明】

- 1、如果调用时，pool 对应的 block 已释放完，pool 将立即被销毁。
- 2、如果调用时，pool 还有未释放的 block，pool 将处于待销毁状态，将不能从 pool 中申请新的 block，待所有 block 都释放完之后，该 pool 将会被销毁。

### ES\_VB\_SetConfig

设置公共视频缓存池属性。

#### 【说明】

- 1、此接口可以重复调用，以最后一次设置的为准。

### ES\_VB\_GetConfig

获取公共视频缓存池属性。

### ES\_VB\_Init

创建公共视频缓存池。

#### 【说明】

- 1、必须先调用 ES\_VB\_SetConfig，否则会返回失败。
- 2、此接口不能重复调用，重复调用返回失败。

### ES\_VB\_Exit

销毁公共视频缓存池。

### ES\_VB\_SetModPoolConfig

设置模块视频缓存池属性。

---

**ES\_VB\_GetModPoolConfig**

获取模块视频缓存池属性。

**ES\_VB\_ModPoolInit**

创建模块视频缓存池。

**【说明】**

1、须先调用 ES\_VB\_SetModConfig，否则会返回失败。

**ES\_VB\_ModPoolExit**

销毁模块视频缓存池。

**ES\_VB\_GetModBlock**

从模块缓存池中获取一个缓存块。

**ES\_VB\_GetBlock**

获取一个缓存块。

**【说明】**

1、用户可以在创建一个缓存池之后，调用本接口从该缓存池中获取一个缓存块；即将第 1 个参数 Pool 设置为创建的缓存池 ID；第 2 个参数 blkSize 须小于或等于创建该缓存池时指定的缓存块大小。从指定缓存池获取缓存块时，参数 strZone 无效。

2、如果用户需要从任意一个公共缓存池中获取一块指定大小的缓存块，则可以将第 1 个参数 PoolId 设置为无效 ID 号 (VB\_INVALID\_POOLID)，将第 2 个参数 blkSize 设置为需要的缓存块大小，并指定要从哪个 strZone 上的公共缓存池获取缓存块。如果指定的 strZone 上并没有公共缓存池，那么将获取不到缓存块。如果 strZone 等于 NULL，则从 mmz\_nid\_0\_part\_0 上的公共缓存池获取缓存块。

**ES\_VB\_ReleaseBlock**

释放一个已经获取的缓存块。

**ES\_VB\_AllocIOVA**

获取指定缓存块对应模块的 IOVA 地址。

**【说明】**

1、ES\_VB\_AllocIOVA 和 ES\_VB\_FreelIOVA 必须成对调用。

2、若是第一次调用 ES\_VB\_AllocIOVA，从 enVbUid 模块获取 iova，则缓存块的引用计数 blockRefCount 会加 1，最后一个成对调用的 ES\_VB\_FreelIOVA 会使 blockRefCount 减 1。

3、此接口正确返回的前提是对应模块的驱动已正确加载。

---

**ES\_VB\_IOVA2Handle**

获取模块 IOVA 地址对应的缓存块 memFd。

**ES\_VB\_FreelIOVA**

释放缓存块的 IOVA 地址空间。

**【说明】**

使用此接口释放 IOVA 之前，必须保证依赖的 memFd 有效，否则无法释放 IOVA，会产生泄露。

**ES\_VB\_Handle2PoolId**

获取一个缓存块所在缓存池的 ID。

**ES\_VB\_MmapPool**

为一个视频缓存池映射用户态虚拟地址。

**【说明】**

- 1、 通过 ES\_VB\_MmapPool 映射得到 pool 用户态虚拟地址，在用户态访问该虚拟地址都是 uncached。

**ES\_VB\_MunmapPool**

为一个视频缓存池解除用户态映射。

**ES\_VB\_GetBlockVirAddr**

获取一个视频缓存池中的缓存块的用户态虚拟地址。

**【说明】**

- 1、 必须已调用 ES\_VB\_MmapPool。

**ES\_VB\_GetBlockRefCount**

获取缓冲块所对应 block 在内核态的引用计数。

**ES\_VB\_registerBlockRefCntNotifyCallback**

对指定 memFd 注册一个 Callback，当引用计数减小到注册指定的 notifyCnt 时， 触发 callback。

**【说明】**

此功能主要是提供：当 dma\_buffer 需要共享给其它任务时，用户可以感知到这些任务已经结束，并唤醒此 notifier 进行下一步工作。

1. ES\_SYS\_SplitMemFd 接口获取的 memFd 不支持注册 callback。
2. 一个 Fd 不支持重复注册 callback
3. 需要跨进程共享的 memFd 不支持注册 callback。



2.3 用户态内存分配管理

内存块来自于用户 malloc 区域，申请的内存块是物理地址连续或非连续的。内存管理主要涉及以下 API：

ES\_USER\_Malloc\_To\_MemFd

将用户态 malloc 内存转换为内存块。

【说明】

- 1、分配的内存来自于用户 malloc 出来的区域。
- 2、调用内存分配接口分配出来的内存 buffer 用一个无符号 64 位类型的 memFd 表示，memFd 在进程内唯一。跨进程需要调用 ES\_SYS\_RcvMemFd 进行转换得到另外进程下对应的 memFd。
- 3、接口内部处理用户态内存对 DMA 操作的对齐要求。
- 4、使用完毕后需要对 malloc 内存进行 free 操作。

3. API 定义

ES\_SYS\_MemAlloc

【函数体】

```
ES_S32 ES_SYS_MemAlloc(  
    ES_U64 *memFd,  
    SYS_CACHE_MODE_E cacheMode,  
    const ES_CHAR *pstrMmb,  
    const ES_CHAR* strZone,  
    ES_U64 len)
```

【描述】

从指定的内存区域分配内存块。

【参数】

| Parameter Name | 描述                                                                                                                                                                                                                 | Input/Output |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| memFd          | 分配内存后返回的 memFd                                                                                                                                                                                                     | Output       |
| cacheMode      | 内存在内核态的 cache 属性<br><br>SYS_CACHE_MODE_NOCACHE: 内核态 CPU 访问时，不经过 CPU 的 cache<br><br>SYS_CACHE_MODE_CACHED: 内核态 CPU 访问时，经过 CPU 的 cache<br><br>SYS_CACHE_MODE_LLC: 内核态 CPU 访问时，经过 LLC, 不经过 CPU 的 cache; 设备通过 dma_addr | Input        |

| Parameter Name | 描述                                                                                                                                                      | Input/Output |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
|                | 访问时，经过 LLC                                                                                                                                              |              |
| pstrMmb        | Mmb 名称的字符串，用于 proc 时的信息呈现                                                                                                                               | Input        |
| strZone        | 分配内存所对应的区域。strZone 可以是 Mmz 的 partition，例如 "mmz_nid_0_part_0"，也可以是 "system", "linux,cma", 或者 "llc_spram0" 来自 SPRAM。若传入 NULL 空指针，则默认使用 "mmz_nid_0_part_0" | Input        |
| len            | 内存大小                                                                                                                                                    | Input        |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_SYS\_Mmap

## 【函数体】

```
ES_VOID* ES_SYS_Mmap(
    ES\_U64 memFd,
    ES\_U64 size,
    SYS\_CACHE\_MODE\_E cacheMode)
```

## 【描述】

映射指定内存块，得到用户态虚拟地址。

## 【参数】

| Parameter Name | 描述                                                                                                                                                                | Input/Output |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| memFd          | 需要映射的 memFd                                                                                                                                                       | Input        |
| size           | 需要映射的内存大小                                                                                                                                                         | Input        |
| cacheMode      | 内存在用户态的 cache 属性<br>SYS_CACHE_MODE_NOCACHE: 用户态 CPU 访问时，不经过 CPU 的 cache<br>SYS_CACHE_MODE_CACHED: 用户态 CPU 访问时，经过 CPU 的 cache<br>SYS_CACHE_MODE_LLC: 用户态 CPU 访问时，经 | Input        |

| Parameter Name | 描述                    | Input/Output |
|----------------|-----------------------|--------------|
|                | 过 LLC，不经过 CPU 的 cache |              |

## 【返回值】

| Return Value | 描述      |
|--------------|---------|
| 0            | Fail    |
| None 0       | 用户态虚拟地址 |

## 【注意】

## ES\_SYS\_Munmap

## 【函数体】

```
ES_S32 ES_SYS_Munmap(
    ES_VOID *pVirAddr,
    ES_U64 size)
```

## 【描述】

反映射指定内存块用户态虚拟地址。

## 【参数】

| Parameter Name | 描述                             | Input/Output |
|----------------|--------------------------------|--------------|
| pVirAddr       | 待操作内存所对应的用户态虚拟地址               | Input        |
| size           | 内存大小：通过 ES_SYS_Mmap 函数 map 的大小 | Input        |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_SYS\_MemFlushCache

## 【函数体】

```
ES_S32 ES_SYS_MemFlushCache(
    ES_U64 memFd)
```

## 【描述】

刷新 cache 里的内容到内存并使 cache 里的内容无效。

【参数】

| Parameter Name | 描述              | Input/Output |
|----------------|-----------------|--------------|
| memFd          | 待操作内存所对应的 memFd | Input        |

【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

【注意】

ES\_SYS\_MemFree

【函数体】

```
ES_S32 ES_SYS_MemFree(  
    ES\_U64 memFd)
```

【描述】

释放通过 ES\_SYS\_MemAlloc 申请得到的内存块。

【参数】

| Parameter Name | 描述         | Input/Output |
|----------------|------------|--------------|
| memFd          | 内存块的 memFd | Input        |

【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

【注意】

ES\_SYS\_SplitMemFd

【函数体】

```
ES_S32 ES_SYS_SplitMemFd(  
    ES\_U64 memFd,  
    ES\_U64 offset,
```

[ES\\_U64](#) len,  
[ES\\_U64](#) \*splittedMemFd)

## 【描述】

从一个内存块的 offset 偏移处，按照指定的 len 大小，生成一个新的内存块。

## 【参数】

| Parameter Name | 描述                                      | Input/Output |
|----------------|-----------------------------------------|--------------|
| memFd          | 缓存块的 memFd                              | Input        |
| offset         | splittedFd 在缓存块的偏移地址，需按 4K PAGE_SIZE 对齐 | Input        |
| len            | splittedFD 的大小，按 PAGE_SIZE 对齐           | Input        |
| splittedMemFd  | 返回 split 的 memFd                        | Output       |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_SYS\_RcvMemFd

## 【函数体】

ES\_S32 ES\_SYS\_RcvMemFd(  
[ES\\_U64](#) sndMemFd,  
[ES\\_U64](#) \*rcvdMemFd)

## 【描述】

在不同进程或者同进程不同软件模块之间传递内存块。

## 【参数】

| Parameter Name | 描述           | Input/Output |
|----------------|--------------|--------------|
| sndMemFd       | 发送端的 MemFd   | Input        |
| rcvdMemFd      | 接收端收到的 MemFd | Output       |

## 【返回值】

| Return Value | 描述      |
|--------------|---------|
| 0            | Success |

|        |                                              |
|--------|----------------------------------------------|
| None 0 | Fail, refers to the <a href="#">_ErrCode</a> |
|--------|----------------------------------------------|

## 【注意】

## ES\_SYS\_GetMemSize

## 【函数体】

```
ES_S32 ES_SYS_GetMemSize(
    ES_U64 memFd,
    ES_U64 *size)
```

## 【描述】

获取指定内存块的大小。

## 【参数】

| Parameter Name | 描述         | Input/Output |
|----------------|------------|--------------|
| memFd          | 缓存块的 memFd | Input        |
| size           | 缓冲块大小      | Output       |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_SYS\_RetrieveMemNodeByFd

## 【函数体】

```
ES_S32 ES_SYS_RetrieveMemNodeByFd(
    ES_U64 memFd,
    EIC770X\_LOGICAL\_MEM\_NODE\_E *nid)
```

## 【描述】

获取指定内存块所在的 node ID。

## 【参数】

| Parameter Name | 描述         | Input/Output |
|----------------|------------|--------------|
| memFd          | 缓存块的 memFd | Input        |
| nid            | node ID    | Output       |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_SYS\_RetrieveMemNodeByVaddr

## 【函数体】

```
ES_S32 ES_SYS_RetrieveMemNodeByVaddr(
    void *pVirAddr,
    EIC770X\_LOGICAL\_MEM\_NODE\_E *nid)
```

## 【描述】

通过缓冲块虚拟地址获取缓存块所在的 node ID。

## 【参数】

| Parameter Name | 描述       | Input/Output |
|----------------|----------|--------------|
| pVirAddr       | 缓存块的虚拟地址 | Input        |
| nid            | node ID  | Output       |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_SYS\_MemFlushCacheAllByNid

## 【函数体】

```
ES_S32 ES_SYS_MemFlushCacheAllByNid(EIC770X\_LOGICAL\_MEM\_NODE\_E nid)
```

## 【描述】

将指定 node ID 所有 L3 缓存内容刷入主存并将缓存内容无效。

## 【参数】

| Parameter Name | 描述      | Input/Output |
|----------------|---------|--------------|
| nid            | node ID | Input        |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_SYS\_MemFlushCacheAll

## 【函数体】

ES\_S32 ES\_SYS\_MemFlushCacheAll()

## 【描述】

将所有 L3 缓存内容刷入主存并将缓存内容无效。

## 【参数】

| Parameter Name | 描述 | Input/Output |
|----------------|----|--------------|
|                |    |              |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_VB\_CreatePool

## 【函数体】

ES\_S32 ES\_VB\_CreatePool(  
[VB\\_POOL\\_CONFIG\\_S](#) \*pstVbPoolCfg,  
[VB\\_POOL](#) \*poolId)

## 【描述】

从指定的内存区域创建一个私有视频缓存池。

## 【参数】

| Parameter Name | 描述          | Input/Output |
|----------------|-------------|--------------|
| pstVbPoolCfg   | 缓存池配置属性参数指针 | Input        |
| poolId         | 缓存池 ID      | Input        |



## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_VB\_DestroyPool

## 【函数体】

```
ES_S32 ES_VB_DestroyPool(  
    VB\_POOL poolId)
```

## 【描述】

销毁一个私有视频缓存池。

## 【参数】

| Parameter Name | 描述     | Input/Output |
|----------------|--------|--------------|
| poolId         | 缓存池 ID | Input        |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_VB\_SetConfig

## 【函数体】

```
ES_S32 ES_VB_SetConfig(  
    const VB\_CONFIG\_S *pVbConfig)
```

## 【描述】

设置公共视频缓存池属性。

## 【参数】

| Parameter Name | 描述          | Input/Output |
|----------------|-------------|--------------|
| pVbConfig      | 公共视频缓冲池属性指针 | Input        |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_VB\_GetConfig

## 【函数体】

```
ES_S32 ES_VB_GetConfig(
    VB\_CONFIG\_S *pVbConfig)
```

## 【描述】

获取公共视频缓存池属性。

## 【参数】

| Parameter Name | 描述          | Input/Output |
|----------------|-------------|--------------|
| pVbConfig      | 公共视频缓冲池属性指针 | Output       |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_VB\_Init

## 【函数体】

```
ES_S32 ES_VB_Init (ES_VOID)
```

## 【描述】

创建公共视频缓存池。

## 【参数】

| Parameter Name | 描述 | Input/Output |
|----------------|----|--------------|
|----------------|----|--------------|

## 【返回值】

| Return Value | 描述                                          |
|--------------|---------------------------------------------|
| 0            | Success                                     |
| None 0       | Fail, refers to the <a href="#">ErrCode</a> |

## 【注意】

## ES\_VB\_Exit

## 【函数体】

ES\_S32 ES\_VB\_Exit(ES\_VOID)

## 【描述】

销毁公共视频缓存池。

## 【参数】

| Parameter Name | 描述 | Input/Output |
|----------------|----|--------------|
|----------------|----|--------------|

## 【返回值】

| Return Value | 描述                                          |
|--------------|---------------------------------------------|
| 0            | Success                                     |
| None 0       | Fail, refers to the <a href="#">ErrCode</a> |

## 【注意】

## ES\_VB\_SetModPoolConfig

## 【函数体】

ES\_S32 ES\_VB\_SetModPoolConfig(  
[VB\\_UID\\_E](#) enVbUid,  
[const VB\\_CONFIG\\_S](#) \*pVbConfig)

## 【描述】

设置模块视频缓存池属性。

## 【参数】

| Parameter Name | 描述          | Input/Output |
|----------------|-------------|--------------|
| enVbUid        | 模块 ID       | Input        |
| pVbConfig      | 模块视频缓冲池属性指针 | Input        |

【返回值】

| Return Value | 描述                                          |
|--------------|---------------------------------------------|
| 0            | Success                                     |
| None 0       | Fail, refers to the <a href="#">ErrCode</a> |

【注意】

ES\_VB\_GetModPoolConfig

【函数体】

```
ES_S32 ES_VB_GetModPoolConfig(  
    VB\_UID\_E enVbUid,  
    VB\_CONFIG\_S *pVbConfig)
```

【描述】

获取模块视频缓存池属性。

【参数】

| Parameter Name | 描述          | Input/Output |
|----------------|-------------|--------------|
| enVbUid        | 模块 ID       | Input        |
| pVbConfig      | 模块视频缓冲池属性指针 | Output       |

【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

【注意】

ES\_VB\_ModPoolInit

【函数体】

```
ES_S32 ES_VB_ModPoolInit(  
    VB\_UID\_E enVbUid)
```

## 【描述】

创建模块视频缓存池。

## 【参数】

| Parameter Name | 描述    | Input/Output |
|----------------|-------|--------------|
| enVbUid        | 模块 ID | Input        |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_VB\_ModPoolExit

## 【函数体】

```
ES_S32 ES_VB_ModPoolExit(  
    VB\_UID\_E enVbUid)
```

## 【描述】

销毁模块视频缓存池。

## 【参数】

| Parameter Name | 描述    | Input/Output |
|----------------|-------|--------------|
| enVbUid        | 模块 ID | Input        |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_VB\_GetModBlock

## 【函数体】

```
ES_S32 ES_VB_GetModBlock(  
    VB\_UID\_E enVbUid,
```

```
ES_U64 blkSize,  
const ES_CHAR * strZone,  
ES_U64 *memFd)
```

**【描述】**

从模块缓存池中获取一个缓存块。

**【参数】**

| Parameter Name | 描述                 | Input/Output |
|----------------|--------------------|--------------|
| enVbUid        | 模块 ID              | Input        |
| blkSize        | 缓存块大小，以 byte 为单位   | Input        |
| strZone        | Mmz 的 partition 名字 | Input        |
| memFd          | 缓存块的 memFd         | Output       |

**【返回值】**

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

**【注意】****ES\_VB\_GetBlock****【函数体】**

```
ES_S32 ES_VB_GetBlock(  
    VB\_POOL poolId,  
    ES_U64 blkSize,  
    const ES_CHAR * strZone,  
    ES_U64 *memFd)
```

**【描述】**

获取一个缓存块。

**【参数】**

| Parameter Name | 描述                 | Input/Output |
|----------------|--------------------|--------------|
| poolId         | 缓存池 ID 号           | Input        |
| blkSize        | 缓存块大小，以 byte 为单位   | Input        |
| strZone        | Mmz 的 partition 名字 | Input        |
| memFd          | 缓存块的 memFd         | Output       |

**【返回值】**

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

【注意】

## ES\_VB\_ReleaseBlock

【函数体】

```
ES_S32 ES_VB_ReleaseBlock(
    ES_U64 memFd)
```

【描述】

释放一个已经获取的缓存块。

【参数】

| Parameter Name | 描述         | Input/Output |
|----------------|------------|--------------|
| memFd          | 缓存块的 memFd | Input        |

【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

【注意】

## ES\_VB\_AllocIOVA

【函数体】

```
ES_S32 ES_VB_AllocIOVA(
    ES_U64 memFd,
    VB_UID_E enVbUid,
    ES_VOID **ppIOVA)
```

【描述】

获取一个缓存块的 IOVA 地址。

【参数】

| Parameter Name | 描述         | Input/Output |
|----------------|------------|--------------|
| memFd          | 缓存块的 memFd | Input        |

| Parameter Name | 描述               | Input/Output |
|----------------|------------------|--------------|
| enVbUid        | 模块 ID            | Input        |
| ppIOVA         | 获取的对应模块的 IOVA 地址 | Output       |

【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

【注意】

ES\_VB\_IOVA2Handle

【函数体】

```
ES_S32 ES_VB_IOVA2Handle(  
    VB\_UID\_E enVbUid,  
    ES\_VOID *pIOVA,  
    ES\_U64 *memFd)
```

【描述】

获取模块 IOVA 地址对应的缓存块 memFd。

【参数】

| Parameter Name | 描述                     | Input/Output |
|----------------|------------------------|--------------|
| enVbUid        | 模块 ID                  | Input        |
| pIOVA          | IOVA 地址                | Input        |
| memFd          | 模块 IOVA 地址对应的缓存块 memFd | Output       |

【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

【注意】

ES\_VB\_FreelOVA

【函数体】

```
ES_S32 ES_VB_FreelOVA(  
    VB\_UID\_E enVbUid,  
    ES\_U64 memFd)
```



## 【描述】

释放缓存块的 IOVA 地址空间。

## 【参数】

| Parameter Name | 描述        | Input/Output |
|----------------|-----------|--------------|
| enVbUid        | 模块 ID     | Input        |
| memFd          | 缓存块 memFd | Input        |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_VB\_Handle2PoolId

## 【函数体】

```
ES_S32 ES_VB_Handle2PoolId(  
    ES\_U64 memFd)
```

## 【描述】

获取一个缓存块所在缓存池的 ID。

## 【参数】

| Parameter Name | 描述        | Input/Output |
|----------------|-----------|--------------|
| memFd          | 缓存块 memFd | Input        |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 大于等于 0       | 缓存块所在缓存池的 ID                                 |
| 小于 0         | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_VB\_MmapPool

## 【函数体】

```
ES_S32 ES_VB_MmapPool(  
    VB\_POOL poolId)
```

## 【描述】

为一个视频缓存池映射用户态虚拟地址。

## 【参数】

| Parameter Name | 描述      | Input/Output |
|----------------|---------|--------------|
| poolId         | 缓存池的 ID | Input        |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_VB\_MunmapPool

## 【函数体】

```
ES_S32 ES_VB_MunmapPool (
    VB\_POOL poolId)
```

## 【描述】

为一个视频缓存池解除用户态映射。

## 【参数】

| Parameter Name | 描述      | Input/Output |
|----------------|---------|--------------|
| poolId         | 缓存池的 ID | Input        |

## 【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

## 【注意】

## ES\_VB\_GetBlockVirAddr

## 【函数体】

```
ES_S32 ES_VB_GetBlockVirAddr(
    VB\_POOL poolId,
    ES\_U64 memFd,
```

ES\_VOID \*\*ppVirAddr)

【描述】

获取一个视频缓存池中的缓存块的用户态虚拟地址。

【参数】

| Parameter Name | 描述         | Input/Output |
|----------------|------------|--------------|
| poolId         | 缓存池的 ID    | Input        |
| memFd          | 缓冲块的 memFd | Input        |
| ppVirAddr      | 缓存块的虚拟地址   | Output       |

【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

【注意】

ES\_VB\_GetBlockRefCount

【函数体】

```
ES_S32 ES_VB_GetBlockRefCount(  
    ES_U64 memFd,  
    ES_U64 *blkFileCnt)
```

【描述】

获取缓冲块所对应 block 在内核态的引用计数。

【参数】

| Parameter Name | 描述          | Input/Output |
|----------------|-------------|--------------|
| memFd          | 缓存块 memFd   | Input        |
| blkFileCnt     | Block 的引用计数 | Output       |

【返回值】

| Return Value | 描述                                           |
|--------------|----------------------------------------------|
| 0            | Success                                      |
| None 0       | Fail, refers to the <a href="#">_ErrCode</a> |

【注意】

ES\_VB\_registerBlockRefCntNotifyCallback

【函数体】

```
ES_S32 ES_VB_registerBlockRefCntNotifyCallback (  
    ES_U64 memFd,  
    ES_U64 notifyRefcnt,  
    VB_BlockRefCntCallback callback,  
    void *callback_arg)
```

【描述】

为指定 memFd 注册一个 notifier，当引用计数减小到 notifyRefCnt 时，触发此 notifier。

【参数】

| Parameter Name | 描述                  | Input/Output |
|----------------|---------------------|--------------|
| memFd          | 缓存块 memFd           | Input        |
| notifyRefcnt   | 目标引用计数值             | Input        |
| callback       | 函数指针，需要执行的 notifier | Input        |
| callback_arg   | Callback 的入参        | Input        |

【返回值】

| Return Value | 描述                                            |
|--------------|-----------------------------------------------|
| 0            | Success                                       |
| None 0       | Fail, refers to the <a href="#">__ErrCode</a> |

【注意】

- 1. ES\_SYS\_SplitMemFd 接口获取的 memFd 不支持注册 callback。
- 2. 一个 Fd 不支持重复注册 callback
- 3. 需要跨进程共享的 memFd 不支持注册 callback。

ES\_USER\_Malloc\_To\_MemFd

【函数体】

```
ES_S32 ES_USER_Malloc_To_MemFd(ES_VOID *orig_ptr, ES_U64 length, ES_U64 *dma_fd,  
    SYS_CACHE_MODE_E cacheType)
```

【描述】

将用户态 malloc 的内存转换为内存块。

【参数】

| Parameter Name | 描述                                                                                                                                     | Input/Output |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------|--------------|
| orig_ptr       | 用户使用 malloc 分配内存的地址                                                                                                                    | Input        |
| len            | 内存大小                                                                                                                                   | Input        |
| dma_fd         | 分配内存后返回的 dma_fd                                                                                                                        | Output       |
| cacheMode      | 内存在内核态的 cache 属性<br><br>SYS_CACHE_MODE_NOCACHE: 内核态 CPU 访问时, 不经过 CPU 的 cache<br><br>SYS_CACHE_MODE_CACHED: 内核态 CPU 访问时, 经过 CPU 的 cache | Input        |

【返回值】

| Return Value | 描述                                          |
|--------------|---------------------------------------------|
| 0            | Success                                     |
| None 0       | Fail, refers to the <a href="#">ErrCode</a> |

【注意】

4. 数据类型

相关数据类型、数据结构定义如下：

- [SYS\\_CACHE\\_MODE\\_E](#): 定义缓存块在 kernel 态或者用户态的 cache 属性
- [VB\\_POOL\\_CONFIG\\_S](#): 定义私有缓存池的属性
- [VB\\_CONFIG\\_S](#): 定义公共缓存池或者模块缓存池的配置参数
- [VB\\_UID\\_E](#): 定义视频缓冲池的模块 ID 枚举类型
- [EIC770X\\_LOGICAL\\_MEM\\_NODE\\_E](#): 定义逻辑 node id 编码。

[SYS\\_CACHE\\_MODE\\_E](#)

【说明】

定义缓存块在 kernel 态或者用户态的 cache 属性

【定义】

```
typedef enum esSYS_CACHE_MODE_E
{
    SYS_CACHE_MODE_NOCACHE = 0,
    SYS_CACHE_MODE_CACHED = 1,
    SYS_CACHE_MODE_LLC = 2,
    SYS_CACHE_MODE_BUTT
} SYS\_CACHE\_MODE\_E;
```

## 【成员】

| Member                 | 描述                                                                 |
|------------------------|--------------------------------------------------------------------|
| SYS_CACHE_MODE_NOCACHE | 内核或用户态映射内存地址，CPU 访问时不经过 CPU cache                                  |
| SYS_CACHE_MODE_CACHED  | 内核或用户态映射内存地址，CPU 访问时经过 CPU cache，需要 flush cache 保证 CPU 和设备访问内存的一致性 |
| SYS_CACHE_MODE_LLC     | 内核或用户态映射内存地址，CPU 和设备访问时，都要经过 LLC，不经过 CPU 的 cache                   |

## VB\_POOL

## 【说明】

定义缓存池的类型

## 【定义】

```
typedef ES_S32 VB_POOL;
```

## VB\_POOL\_CONFIG\_S

## 【说明】

定义私有缓存池的属性

## 【定义】

```
typedef struct esVB_POOL_CONFIG_S
{
    ES_U64 blkSize;
    ES_U32 blkCnt;
    SYS_CACHE_MODE_E enRemapMode;
    ES_CHAR mmzName[ES_MAX_MMZ_NAME_LEN];
}VB_POOL_CONFIG_S;
```

## 【成员】

| Member      | 描述                  |
|-------------|---------------------|
| blkSize     | 缓存块大小，以 Byte 位单位    |
| blkCnt      | 每个缓存池的缓存块个数         |
| enRemapMode | VB 的内核态虚拟地址映射模式     |
| mmzName     | 当前缓存池从哪个 MMZ 区域分配内存 |

## VB\_CONFIG\_S

## 【说明】

公共缓存池或者模块缓存池的配置参数

【定义】

```
typedef struct esVB_CONFIG_S
{
    ES_U32 poolCnt;
    VB_POOL_CONFIG_S poolCfgs[L];
}VB_CONFIG_S;
```

【成员】

| Member   | 描述       |
|----------|----------|
| poolCnt  | 配置缓存池的个数 |
| poolCfgs | 缓存池的配置参数 |

VB\_UID\_E

【说明】

视频缓冲池的模块 ID 枚举类型

【定义】

```
typedef enum esVB_UID_E {
    VB_UID_PRIVATE = 0,
    VB_UID_COMMON,
    VB_UID_VI,
    VB_UID_VO,
    VB_UID_VPS,
    VB_UID_VENC,
    VB_UID_VDEC,
    VB_UID_HAE,
    VB_UID_USER,
    VB_UID_BUTT,
    VB_UID_MAX,
} VB_UID_E;
```

EIC770X\_LOGICAL\_MEM\_NODE\_E

【说明】

逻辑 node ID 枚举类型，

【定义】

```
typedef enum {
    EIC770X_LOGICAL_FLAT_MEM_NODE_0 = 0,
    EIC770X_LOGICAL_FLAT_MEM_NODE_1 = 1,
    EIC770X_LOGICAL_SDRAM_NODE_0 = 500,
    EIC770X_LOGICAL_SDRAM_NODE_1 = 501,
    EIC770X_LOGICAL_INTERLEAVE_MEM_NODE = 1000,
} EIC770X_LOGICAL_MEM_NODE_E;
```

【成员】

| Member | 描述 |
|--------|----|
|--------|----|

|                                     |                               |
|-------------------------------------|-------------------------------|
| EIC770X_LOGICAL_FLAT_MEM_NODE_0     | DIE 0 对应的 node ID 逻辑编码        |
| EIC770X_LOGICAL_FLAT_MEM_NODE_1     | DIE 1 对应的 node ID 逻辑编码        |
| EIC770X_LOGICAL_SPRAM_NODE_0        | DIE 0 上 SPRAM 的 node ID 逻辑编码  |
| EIC770X_LOGICAL_SPRAM_NODE_1        | DIE 1 上 SPRAM 的 node ID 逻辑编码  |
| EIC770X_LOGICAL_INTERLEAVE_MEM_NODE | INTERLEAVE 模式 对应 node ID 逻辑编码 |

## 5. ErrCode

表 5-1 Error code

| ErrCode    | Macro Definition              | 描述                                                      |
|------------|-------------------------------|---------------------------------------------------------|
| 0xA0026003 | ES_SYS_ERROR_ILLEGAL_PARAM    | Invalid parameter                                       |
| 0xA0026006 | ES_SYS_ERROR_NULL_PTR         | Using a NULL point                                      |
| 0xA0026007 | ES_SYS_ERROR_NOT_CONFIG       | Try to initialize system, before<br>configing attribute |
| 0xA0026008 | ES_SYS_ERROR_NOT_SUPPORT      | Operation or type is not supported                      |
| 0xA0026009 | ES_SYS_ERROR_NOT_PERM         | Operation is not permitted                              |
| 0xA002600C | ES_SYS_ERROR_NOMEM            | Failure caused by malloc memory                         |
| 0xA002600D | ES_SYS_ERROR_NOBUF            | Failure caused by malloc buffer                         |
| 0xA0026011 | ES_SYS_ERROR_BADADDR          | Bad address                                             |
| 0xA0026013 | ES_SYS_ERROR_SIZE_NOT_ENOUGH  | Buffer size is smaller than the actual<br>size required |
| 0xA0026040 | ES_SYS_ERROR_DEV_NOT_FOUND    | Device not found                                        |
| 0xA0026041 | ES_SYS_ERROR_DEV_NOT_OPEN     | Device not open                                         |
| 0xA0026042 | ES_SYS_ERROR_OPERATION_FAIL   | Operation failed                                        |
| 0xA0026043 | ES_SYS_ERROR_FLUSH_CACHE_FAIL | Failed to flush cache                                   |
| 0xA0026044 | ES_SYS_ERROR_MEM_FREE_FAIL    | Failed to free memory                                   |
| 0xA0026045 | ES_SYS_ERROR_GET_SIZE_FAIL    | Failed to get size                                      |
| 0xA0016003 | ES_VB_ERROR_ILLEGAL_PARAM     | Invalid parameter                                       |
| 0xA0016006 | ES_VB_ERROR_NULL_PTR          | Using a NULL point                                      |
| 0xA0016007 | ES_VB_ERROR_NOT_CONFIG        | Try to initialize system, before<br>configing attribute |
| 0xA0016008 | ES_VB_ERROR_NOT_SUPPORT       | Operation or type is not supported                      |



|            |                                 |                                                             |
|------------|---------------------------------|-------------------------------------------------------------|
| 0xA0016009 | ES_VB_ERROR_NOT_PERM            | Operation is not permitted                                  |
| 0xA001600C | ES_VB_ERROR_NOMEM               | Failure caused by malloc memory                             |
| 0xA001600D | ES_VB_ERROR_NOBUF               | Failure caused by malloc buffer                             |
| 0xA0016011 | ES_VB_ERROR_BADADDR             | Bad address                                                 |
| 0xA0016013 | ES_VB_ERROR_SIZE_NOT_ENOUGH     | Buffer size is smaller than the actual size required        |
| 0xA0016040 | ES_VB_ERROR_DEV_NOT_FOUND       | Device not found                                            |
| 0xA0016041 | ES_VB_ERROR_DEV_NOT_OPEN        | Device not open                                             |
| 0xA0016042 | ES_VB_ERROR_OPERATION_FAIL      | Operation failed                                            |
| 0xA0016043 | ES_VB_ERROR_FLUSH_CACHE_FAIL    | Failed to flush cache                                       |
| 0xA0016044 | ES_VB_ERROR_CREAT_POOL_FAIL     | Failed to crate memory pool                                 |
| 0xA0016045 | ES_VB_ERROR_DESTROY_POOL_FAIL   | Failed to destroy memory pool                               |
| 0xA0016046 | ES_VB_ERROR_SET_CFG_FAIL        | Failed to set the configuration of mod pool                 |
| 0xA0016047 | ES_VB_ERROR_GET_CFG_FAIL        | Failed to get the configuration of mod pool                 |
| 0xA0016048 | ES_VB_ERROR_INIT_CFG_FAIL       | Failed to initialize the mod pool                           |
| 0xA0016049 | ES_VB_ERROR_UNINIT_CFG_FAIL     | Failed to uninitialize the mod pool                         |
| 0xA001604A | ES_VB_ERROR_POOL_GET_SIZE_FAIL  | Failed to get the size of pool                              |
| 0xA001604B | ES_VB_ERROR_POOL_MMAP_FAIL      | Failed to map pool                                          |
| 0xA001604C | ES_VB_ERROR_POOL_ILLEGAL_POOLID | Invalid poolId                                              |
| 0xA001604D | ES_VB_ERROR_POOL_UNMAP_FAIL     | Failed to unmap pool                                        |
| 0xA001604E | ES_VB_ERROR_POOL_NOT_MAP        | Failed to get block virtual address when pool is not mapped |
| 0xA001604F | ES_VB_ERROR_GET_IOVA_FAIL       | Failed to get IOVA for specific hardware module             |
| 0xA0016050 | ES_VB_ERROR_PUT_IOVA_FAIL       | Failed to put IOVA for specific hardware module             |
| 0xA0016051 | ES_VB_ERROR_SPLIT_DMABUF_FAIL   | Failed to split memFd                                       |
| 0xA0016052 | ES_VB_ERROR_GET_BLK_FAIL        | Failed to get a block from driver                           |
| 0xA0016053 | ES_VB_ERROR_GET_BLK_OFFSET_FAIL | Failed to get the offset for block in pool                  |
| 0xA0016054 | ES_VB_ERROR_FLUSH_POOL_FAIL     | Failed to flush cache for pool                              |
| 0xA0016055 | ES_VB_ERROR_GET_BLK_REFCNT_FAIL | Failed to get the reference count for block                 |

## 6. 附录

### 使用说明

- 1、使用前确保板端运行环境下交付件中所有驱动 ko 文件已正常加载
- 2、在板端运行环境下/usr/local/lib 下存在动态库 libes\_memory.so
- 3、各 API 的使用请参考示例代码 memory\_samples.c

### Proc 调试信息说明

调试信息采用了 Linux 下的 proc 文件系统，可实时反映当前系统的运行状态，所记录的信息可供问题定位及分析时使用。

#### 1、proc/eswin/vb

在板端环境命令行下执行 `cat /proc/eswin/vb`，可以显示所有 MMZ 区域当前的分配和使用情况。例如在 `mmz_nid_0_part_0` 区域创建一个包含 4 个 block 的 pool，已申请了一个 block 的示例如下图：

```
eswin@rockos-eswin:~$ cat /proc/eswin/vb
Module: [VB], Build Time[xx]
-----MMZ REGION CONFIG-----
      memblock: mmz_nid_0_part_0, total size(0x17ffff000), free mem size(0x17ffff000)
-----POOL CONFIG-----
eswin@rockos-eswin:~$
```

#### 2、proc/llc\_spram0\_info

在板端环境命令行下执行 `cat /proc/llc_spram0_info`，可以显示 SPRAM 的总大小，及当前使用情况。

```
# cat /proc/llc_spram0_info
[root@debian:~]$ cat /proc/llc_spram0_info
SRAM pool: 4096 KiB, Available: 4096 KiB
LLC Users(0):
[root@debian:~]$
```

#### 3、查看所有分配的 dmabuf 信息

在板端环境命令行下顺序执行以下两个命令，可以查看所有分配的 dmabuf 信息。

```
# mount -t debugfs none /sys/kernel/debug
# cat /sys/kernel/debug/dma_buf/bufinfo
```

例如当前有四个 dambuf 正在被使用的情况如下图：

```
[root@debian:~]$ mount -t debugfs none /sys/kernel/debug
[root@debian:~]$ cat /sys/kernel/debug/dma_buf/bufinfo

Dma-buf Objects:
size      flags      mode      count      exp_name      ino      name
01572864  00000002  00080007  00000002  mmz_nid_0_part_0  00000084  <none>
    Attached Devices:
    50430000.dma-controller-hsp
Total 1 devices attached

03145728  00000002  00080007  00000004  mmz_nid_0_part_0  00000083  dst
    Attached Devices:
    50430000.dma-controller-hsp
Total 1 devices attached

01572864  00000002  00080007  00000002  llc_spram0  00000082  <none>
    Attached Devices:
    50430000.dma-controller-hsp
Total 1 devices attached

03145728  00000002  00080007  00000004  llc_spram0  00000081  src
    Attached Devices:
    50430000.dma-controller-hsp
Total 1 devices attached

Total 4 objects, 9437184 bytes
[root@debian:~]$
```